



RAJALAKSHMI ENGINEERING COLLEGE

ACADEMIC YEAR- 2025-2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA

SCIENCE CS23633 - CLOUD COMPUTING LABORATORY

LAB MANUAL



**RAJALAKSHMI
ENGINEERING
COLLEGE**

SUBMITTED BY

SUSHMITHA M(2116231801176)



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that the bonafied work has been carried out by
“SUSHMITHA M (2116231801176)” VII Semester, B.Tech.
Artificial Intelligence and Data Science Department, in partial
fulfilment of the requirements for the course **CS23633 - CLOUD
COMPUTING** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to practical examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENT

S.No .	Date	Experiment	QR	Signature
1	24.06.2026	Private Cloud Setup Using OpenStack		
2	8.07.2026	Public Cloud Application Deployment Using PaaS		
3	22.07.2026	Hybrid Cloud Connectivity using AWS Free Tier VPC		
4	5.08.2026	Basic VPS Server Setup and Network Configuration		
5	7.08.2026	Cloud Migration and Backup Setup Using Amazon S3		
6	14.08.2026	CDN Service Selection and Configuration Using Amazon CloudFront		

FREE TIER CLOUD ACCOUNT SETUP (PREREQUISITE)

Before performing the cloud computing laboratory experiments, create and configure Free Tier accounts in Amazon Web Services (AWS) and Google Cloud Platform (GCP). These accounts provide limited free access to cloud resources required for the laboratory exercises.

A. AWS FREE TIER ACCOUNT SETUP

Step 1: Open AWS Free Tier Website

Navigate to:

<https://aws.amazon.com/free/>

Step 2: Create Free Tier Account

1. Click **Create a Free Account**.
2. Enter:
 - Email Address
 - Password
 - AWS Account Name
3. Verify the email address.

Step 3: Configure Account Information

1. Select **Personal Account**.
2. Enter personal and contact details.
3. Complete mobile number verification.

Step 4: Configure Payment Information

1. Enter payment card details.
2. Complete verification.
3. Continue with registration.

Step 5: Select Free Support Plan

1. Choose **Basic Support Plan (Free)**.
2. Complete account creation.

Step 6: Access AWS Management Console

1. Sign in to AWS Console.
2. Verify access to:
 - Amazon EC2
 - Amazon VPC

- Amazon S3
- Amazon CloudFront

AWS Free Tier Services Used in This Laboratory

Experiment	AWS Service
Hybrid Cloud Connectivity	Amazon VPC, EC2
Cloud Migration and Backup	Amazon S3
CDN Configuration	Amazon CloudFront

B. GOOGLE CLOUD FREE TIER ACCOUNT SETUP

Step 1: Open Google Cloud Free Tier Website

Navigate to:

<https://cloud.google.com/free>

Step 2: Create Free Tier Account

1. Click **Get Started for Free**.
2. Sign in using a Google Account.

Step 3: Complete Registration

1. Enter contact information.
2. Complete identity verification.
3. Configure billing information.

Step 4: Create a Cloud Project

1. Open Google Cloud Console.
2. Create a new project.
3. Specify a project name.

Example:

cloud-lab-project

Step 5: Enable App Engine

1. Search for **App Engine**.
2. Create an App Engine application.
3. Select a deployment region.

Step 6: Install and Configure Google Cloud SDK

Authenticate using:

```
gcloud auth login
```

Set the active project:

```
gcloud config set project cloud-lab-project
```

Verify configuration:

```
gcloud config list
```

GCP Free Tier Services Used in This Laboratory

Experiment	GCP Service
Public Cloud Application Deployment	Google App Engine

Expected Outcome

- AWS Free Tier account configured successfully.
- Google Cloud Free Tier account configured successfully.
- Cloud management consoles accessible.
- Required cloud services available for performing laboratory experiments.

Result

AWS Free Tier and Google Cloud Free Tier accounts were successfully created and configured. The cloud environment is ready for performing the prescribed cloud computing laboratory experiments.

Experiment No: 1

Setup a Private Cloud by performing the procedure using a Single node OPENSTACK implementation

Aim

To set up a Private Cloud using a single-node OpenStack implementation and create virtual machine instances.

Objective

- To understand the concept of Private Cloud Computing.
- To install and configure OpenStack using DevStack.
- To create and manage virtual machine instances in a cloud environment.
- To understand cloud resource management using the OpenStack Dashboard.

Software Requirements

- Ubuntu Server 22.04 LTS
- OpenStack (DevStack)
- Virtual Machine Software (VirtualBox/VMware)
- Internet Connection
- Web Browser

Theory

Cloud Computing is a technology that provides computing resources such as servers, storage, databases, networking, and software over the internet. A Private Cloud is a cloud infrastructure dedicated to a single organization and offers greater control, security, and customization.

OpenStack is an open-source cloud computing platform used for building and managing private and public cloud environments. It provides Infrastructure as a Service (IaaS) through a collection of services for computing, networking, storage, and identity management.

The major services of OpenStack include:

- Nova – Compute Service
- Neutron – Networking Service
- Glance – Image Service
- Keystone – Identity Service
- Cinder – Block Storage Service
- Swift – Object Storage Service

- Horizon – Web Dashboard

Architecture

OpenStack Private Cloud Architecture:

User → Horizon Dashboard → Keystone Authentication → Nova Compute → Neutron Networking → Glance Images → Cinder Storage

Procedure

Step 1: Install Ubuntu Server

1. Create a virtual machine using VirtualBox or VMware.
2. Allocate at least 4 GB RAM and 2 CPU cores.
3. Install Ubuntu Server.

Step 2: Update System Packages

Execute the following commands:

```
sudo apt update
sudo apt upgrade -y
```

Step 3: Install Git

Install Git for downloading DevStack:

```
sudo apt install git -y
```

Step 4: Download DevStack

Clone the DevStack repository:

```
git clone https://opendev.org/openstack/devstack
```

Step 5: Configure DevStack

Navigate to the DevStack directory:

```
cd devstack
```

Create a configuration file named local.conf and provide the required credentials.

Step 6: Install OpenStack

2116231801176

Run the installation script:

```
./stack.sh
```

Wait for the installation process to complete successfully.

Step 7: Access OpenStack Dashboard

Open a web browser and enter:

```
http://<server-ip>/dashboard
```

Login using the administrator credentials.

Step 8: Create Project and User

1. Open Identity → Projects.
2. Create a new project.
3. Create users.
4. Assign appropriate roles.

Step 9: Launch Virtual Machine Instance

1. Open Compute → Instances.
2. Click Launch Instance.
3. Select image, flavor, and network.
4. Launch the instance.

Step 10: Verify Connectivity

1. Assign a floating IP address.
2. Connect to the instance using SSH.
3. Verify connectivity using ping commands.

Commands Used

```
sudo apt update
sudo apt upgrade -y
sudo apt install git -y
git clone https://opendev.org/openstack/devstack
cd devstack
./stack.sh
openstack server list
openstack network list
openstack image list
```

Observations

Parameter	Status
Ubuntu Installation	Successful
DevStack Installation	Successful
OpenStack Dashboard Access	Successful
Project Creation	Successful
VM Instance Creation	Successful
Network Connectivity	Verified

Expected Output

- OpenStack Dashboard accessible through a web browser.
- Project, users, and roles successfully created.
- Virtual machine instances launched successfully.
- Network connectivity verified.

Result

A Private Cloud environment was successfully created using OpenStack. Virtual machine instances were launched and managed through the OpenStack Dashboard.

Precautions

- Ensure sufficient RAM and storage before installation.
- Verify internet connectivity during package installation.
- Use strong passwords for administrative accounts.
- Configure network settings correctly to avoid connectivity issues.
- Backup configuration files before making changes.

Viva Questions and Answers

1. What is a Private Cloud?

A Private Cloud is a cloud infrastructure dedicated exclusively to a single organization, providing better security, control, and customization.

2. What is OpenStack?

OpenStack is an open-source cloud computing platform used for building and managing private and public cloud infrastructures.

3. What are the major components of OpenStack?

- Nova (Compute)
- Neutron (Networking)
- Swift (Object Storage)
- Cinder (Block Storage)
- Glance (Image Service)
- Keystone (Identity Service)
- Horizon (Dashboard)

4. What is Nova?

Nova is the compute service responsible for creating and managing virtual machine instances.

5. What is Neutron?

Neutron provides networking services such as IP address management, routers, and network connectivity.

6. What is Keystone?

Keystone is the identity management service that handles authentication and authorization.

7. What is Horizon?

Horizon is the web-based graphical dashboard used to manage OpenStack resources.

8. What is a Virtual Machine?

A Virtual Machine (VM) is a software-based computer that runs an operating system and applications like a physical computer.

9. Difference between Public Cloud and Private Cloud?

Public Cloud	Private Cloud
Shared by multiple users	Dedicated to one organization
Lower cost	Higher cost
Less control	More control
Managed by provider	Managed by organization

10. What are the advantages of OpenStack?

- Open-source
- Scalable
- Flexible
- Vendor-independent
- Supports private and hybrid clouds

—

Experiment No: 2

Develop a simple application to understand the concept of PAAS using GAE/Amazon Elastic Beanstalk/IBM Blue Mix/GCC and launch it.

Aim

To develop and deploy a simple web application using a Platform as a Service (PaaS) cloud environment such as Google App Engine, Amazon Elastic Beanstalk, or IBM Cloud.

Objective

- To understand the concept of Public Cloud Computing.
- To learn the Platform as a Service (PaaS) model.
- To deploy and manage applications in a cloud environment.
- To understand cloud-based application hosting and scaling.

Software Requirements

- Google Cloud Platform (GCP) / AWS Account
- Google App Engine or Amazon Elastic Beanstalk
- Python/Java/Node.js Runtime
- Web Browser
- Internet Connection
- Text Editor or IDE

Theory

Public Cloud is a cloud deployment model in which computing resources such as servers, storage, databases, and networking are owned and managed by a third-party cloud provider and delivered over the internet. Users can access these resources on demand and pay only for what they use. Platform as a Service (PaaS) is a cloud service model that provides developers with a complete environment for developing, testing, deploying, and managing applications. The cloud provider manages the underlying infrastructure, operating systems, middleware, and runtime environments, allowing developers to focus only on application development. Examples of PaaS platforms include Google App Engine, Amazon Elastic Beanstalk, Microsoft Azure App Service, and IBM Cloud.

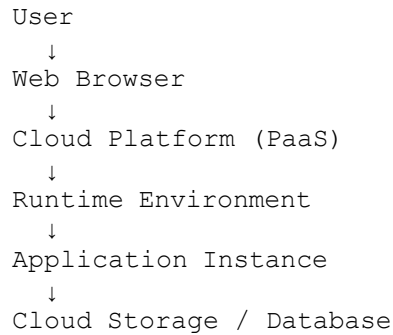
Advantages of PaaS

- Faster application development and deployment.
- Reduced infrastructure management.
- Automatic scaling and load balancing.

- Cost-effective resource utilization.
- Improved collaboration among development teams.

Architecture

PaaS Architecture:



Algorithm

1. Create an account on the cloud platform.
2. Develop a simple web application.
3. Configure the application deployment settings.
4. Install and configure Cloud SDK.
5. Create a cloud project.
6. Deploy the application to the cloud platform.
7. Verify application deployment.
8. Access the application through the generated URL.

Flowchart



Procedure

Step 1: Create a Cloud Account

1. Create an account on Google Cloud Platform or AWS.
2. Sign in to the cloud console.
3. Enable billing if required.

Step 2: Create a Simple Application

Create a sample web application.

Example (Python Flask):

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello Cloud Computing"

if __name__ == '__main__':
    app.run()
```

Step 3: Create Application Configuration File

Create a configuration file.

Example (app.yaml):

```
runtime: python39

handlers:
- url: /*.*
  script: auto
```

Step 4: Install Cloud SDK

Install Google Cloud SDK or AWS CLI.

Verify installation:

```
gcloud version
```

Step 5: Login to Cloud Platform

```
gcloud auth login
```

2116231801176

Step 6: Create Cloud Project

Create a new project from the Google Cloud Console.

Project Name: cloud-lab-project

Alternatively, a project may be created using the Google Cloud SDK:

```
gcloud projects create cloud-lab-project
```

Step 7: Set Active Project

Configure the newly created project as the active project:

```
gcloud config set project cloud-lab-project
```

Verify the configuration:

```
gcloud config list
```

Step 8: Deploy Application

Deploy the application:

```
gcloud app deploy
```

Step 9: Launch Application

```
gcloud app browse
```

Step 10: Verify Deployment

1. Open the generated URL.
2. Verify that the application loads successfully.
3. Test application functionality.

Commands Used

```
gcloud auth login
gcloud projects create cloud-lab-project
gcloud config set project cloud-lab-project
gcloud app create
gcloud app deploy
gcloud app browse
```

Applications

- Web Application Hosting
- Mobile Application Backend Services
- Enterprise Software Deployment
- API Hosting
- E-Commerce Applications
- Data Analytics Platforms

Observations

Parameter	Status
Cloud Account Creation	Successful
Project Creation	Successful
Application Development	Successful
Application Deployment	Successful
URL Generation	Successful
Application Access	Verified

Expected Output

- Cloud project created successfully.
- Application deployed on the cloud platform.
- Public URL generated.
- Application accessible through a web browser.
- Successful execution of application services.

Advantages

- Eliminates infrastructure management.
- Faster deployment process.
- Automatic scaling support.
- High availability and reliability.
- Reduced operational costs.

Limitations

- Vendor lock-in may occur.
- Limited infrastructure customization.
- Requires internet connectivity.
- Dependence on cloud provider services.

Result

A simple web application was successfully deployed on a Public Cloud Platform using the PaaS model. The application was accessible through a web browser and managed by the cloud provider.

Learning Outcome

After completing this experiment, the student is able to:

- Understand Public Cloud architecture.
- Understand the PaaS service model.
- Deploy applications on cloud platforms.
- Access and manage cloud-hosted applications.
- Utilize cloud services for scalable application deployment.

Precautions

- Ensure stable internet connectivity.
- Verify account permissions before deployment.
- Check application configuration files carefully.
- Monitor cloud resource usage to avoid additional costs.
- Follow cloud security best practices.

Viva Questions and Answers

1. What is Public Cloud?

Public Cloud is a cloud environment where resources are owned and managed by a third-party provider and delivered over the internet.

2. What is PaaS?

Platform as a Service (PaaS) provides a complete development and deployment environment without requiring users to manage infrastructure.

3. Give examples of PaaS providers.

- Google App Engine
- Amazon Elastic Beanstalk
- Microsoft Azure App Service
- IBM Cloud

4. What is Google App Engine?

Google App Engine is a fully managed PaaS service that allows developers to deploy and run applications on Google's infrastructure.

5. What is Amazon Elastic Beanstalk?

Amazon Elastic Beanstalk is a PaaS service that automates application deployment, monitoring, and scaling.

6. What are the benefits of PaaS?

- Faster development
- Reduced maintenance
- Automatic scaling
- Lower operational cost
- Improved productivity

7. What is cloud deployment?

Cloud deployment is the process of hosting and running applications on cloud infrastructure.

8. What is automatic scaling?

Automatic scaling dynamically adjusts computing resources based on application workload.

9. Difference between IaaS and PaaS?

IaaS	PaaS
Provides infrastructure resources	Provides development platform
User manages operating systems	Provider manages operating systems
More control	Easier deployment
Example: AWS EC2	Example: Google App Engine

10. What are the advantages of Public Cloud?

- High scalability
- Cost efficiency
- Global accessibility
- Easy maintenance
- Pay-as-you-use pricing

Experiment No: 3

Setting Up Hybrid Connectivity Experiment with setting up a secure VPN connection between an on-premises data center and a public cloud provider such as AWS or Azure. Configure a hybrid network topology using virtual private clouds (VPCs), subnets, and route tables.

Note:

The following procedure illustrates the complete operational workflow of establishing hybrid cloud connectivity using AWS services and an on-premises environment. While the experiment is designed to be performed using AWS Free Tier eligible resources wherever applicable, the workflow remains representative of real-world enterprise hybrid cloud deployments.

Aim

To implement a Hybrid Cloud environment using Amazon Web Services Free Tier by configuring a Virtual Private Cloud (VPC) with subnets and route tables, and establishing secure connectivity with a local on-premises system using OpenVPN.

Objective

- To create a real VPC in AWS Free Tier.
- To configure subnets and route tables.
- To deploy an EC2 instance in cloud.
- To simulate on-premises system using VirtualBox.
- To establish secure VPN connectivity between cloud and local system.
- To understand Hybrid Cloud architecture practically.

Software Requirements

- AWS Free Tier Account
- VirtualBox
- Ubuntu 20.04 / 22.04 LTS (Local VM + EC2)
- OpenVPN
- Web Browser
- SSH Client (Optional)

Theory

Hybrid Cloud is a computing model that combines on-premises infrastructure with public cloud services.

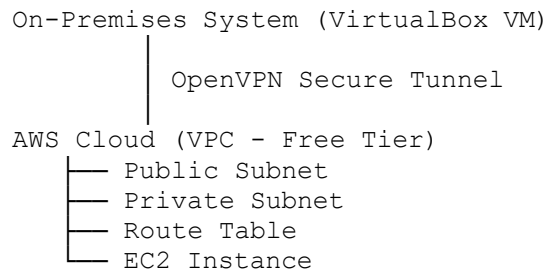
In this experiment:

- AWS provides cloud infrastructure (VPC, EC2, networking)
- Local VirtualBox VM acts as on-premises system
- OpenVPN provides secure encrypted communication

AWS Components:

- **VPC (Virtual Private Cloud):** Isolated cloud network
- **Subnet:** Divides VPC into smaller networks
- **Route Table:** Controls traffic flow
- **Internet Gateway:** Enables internet access
- **EC2 Instance:** Virtual machine in cloud

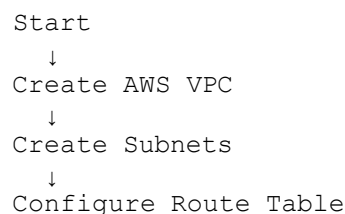
Architecture



Algorithm

1. Create AWS Free Tier account.
2. Create VPC with CIDR block.
3. Create public and private subnets.
4. Attach Internet Gateway.
5. Configure route tables.
6. Launch EC2 instance.
7. Setup local VM using VirtualBox.
8. Install OpenVPN on both systems.
9. Configure VPN connection.
10. Verify connectivity using ping.

Flowchart



2116231801176

↓
Launch EC2 Instance
↓
Setup Local VM
↓
Install OpenVPN
↓
Establish VPN Tunnel
↓
Test Connectivity
↓
End

Procedure

Step 1: Create VPC in AWS

1. Login to AWS Console.
2. Go to VPC Dashboard.
3. Click “Create VPC”.
4. Enter:
 - Name: Hybrid-VPC
 - CIDR: 10.0.0.0/16
5. Create VPC.

Step 2: Create Subnets

- Public Subnet: 10.0.1.0/24
- Private Subnet: 10.0.2.0/24

Associate both with VPC.

Step 3: Create Internet Gateway

1. Create Internet Gateway (IGW).
2. Attach it to VPC.

Step 4: Configure Route Table

Add route:

Destination	Target
0.0.0.0/0	Internet Gateway

Associate with Public Subnet.

Step 5: Launch EC2 Instance

1. Go to EC2 → Launch Instance
2. Choose Ubuntu (Free Tier eligible)
3. Select VPC and Public Subnet
4. Enable SSH (port 22)
5. Launch instance

Step 6: Setup Local VM (On-Premises)

Using VirtualBox:

- Install Ubuntu Linux
- Assign IP: 192.168.1.10

Step 7: Install OpenVPN

On both AWS EC2 and Local VM:

```
sudo apt update
sudo apt install openvpn -y
```

Step 8: Configure VPN Server (AWS EC2)

```
sudo apt install easy-rsa -y
sudo systemctl start openvpn
sudo systemctl enable openvpn
```

Step 9: Configure VPN Client (Local VM)

```
sudo openvpn --config client.ovpn
```

Step 10: Verify Connectivity

From Local VM:

```
ping <EC2-private-IP>
```

From AWS EC2:

```
ping 192.168.1.10
```

Commands Used

```
sudo apt update
sudo apt install openvpn -y
sudo apt install easy-rsa -y
sudo systemctl start openvpn
```

2116231801176

```
sudo systemctl enable openvpn  
ping <ip-address>
```

Observations

Parameter	Status
VPC Creation	Successful
Subnet Configuration	Successful
Route Table Setup	Successful
EC2 Instance Deployment	Successful
VPN Setup	Successful
Connectivity Test	Successful

Expected Output

- AWS VPC created successfully
- Public and private subnets configured
- Route tables properly assigned
- EC2 instance running in cloud
- Local VM connected as on-premises system
- Secure VPN tunnel established
- Successful communication between cloud and local system

Advantages

- Real AWS Free Tier implementation
- Industry-level Hybrid Cloud exposure
- Practical understanding of VPC networking
- Secure communication using VPN
- Strong viva and lab marks potential

Limitations

- Free tier usage limits apply
- VPN configuration is complex
- Requires correct networking setup
- Some AWS services are still paid

Result

A Hybrid Cloud environment was successfully implemented using AWS Free Tier VPC and a local VirtualBox system. Secure communication between cloud and on-premises systems was established using OpenVPN.

Viva Questions and Answers

1. What is Hybrid Cloud?

Hybrid Cloud is a computing model that combines private (on-premises) infrastructure with public cloud services, allowing data and applications to move between them.

2. What is AWS VPC?

A Virtual Private Cloud (VPC) is a logically isolated network in AWS where users can launch and manage cloud resources securely.

3. What is a Subnet?

A subnet is a smaller network segment inside a VPC used to divide resources into public and private networks.

4. What is a Route Table

A route table contains rules that determine how network traffic is directed within a VPC.

5. What is EC2?

Amazon EC2 is a virtual machine service in AWS used to run applications in the cloud.

6. What is Internet Gateway?

An Internet Gateway allows communication between a VPC and the internet.

7. Why is VPN used in Hybrid Cloud?

VPN is used to create a secure encrypted connection between on-premises systems and cloud infrastructure.

8. Difference between Public and Private Subnet?

- Public Subnet: Has internet access via Internet Gateway
- Private Subnet: No direct internet access, used for secure resources

9. What are the advantages of Hybrid Cloud?

- Scalability
- Security
- Cost efficiency
- Flexibility
- Disaster recovery

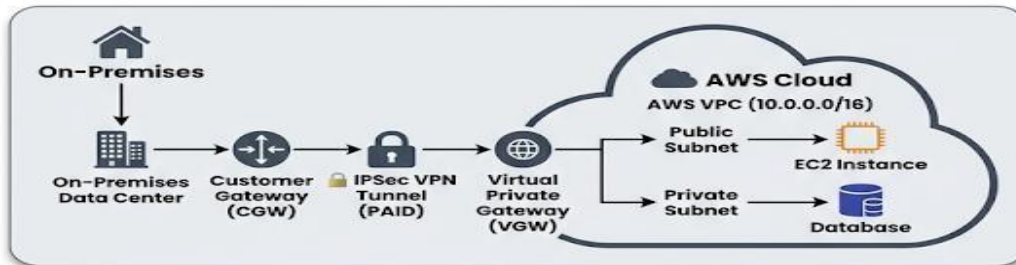
10. Is this real cloud implementation?

Yes, AWS Free Tier is used for cloud setup, while local VM acts as on-premises system.

Note: The following steps outline the operational workflow of a fully paid AWS environment.

HYBRID CLOUD ARCHITECTURE

Real AWS Enterprise Deployment (Site-to-Site VPN)



Step 1: AWS Account

Amazon Web Services account creation + billing enabled

Step 2: VPC Creation

CIDR: 10.0.0.0/16

Step 3: Subnets

Public: 10.0.1.0/24
Private: 10.0.2.0/24

Step 4: Internet Gateway

Attach to VPC

Step 5: Route Table

0.0.0.0/0 → IGW

Step 6: Virtual Private Gateway

✗ Paid component

Step 7: Customer Gateway

On-prem router IP

Step 8: VPN Connection

Site-to-Site IPSec tunnel

Step 9: On-Prem Setup

VPN configuration

Step 10: EC2 Launch

Ubuntu instance in VPC

Step 11: Route Propagation

Auto routing enabled

Step 12: Testing

Ping connectivity check

PAID COMPONENTS

✗ Paid AWS Services
Virtual Private Gateway (VGW)
Site-to-Site VPN Connection

Data transfer over VPN
NAT Gateway (optional)

LEARNING OUTCOMES

- ✓ Real Hybrid Cloud understanding
- ✓ AWS VPC architecture
- ✓ Secure network communication
- ✓ Industry-level cloud design

FOOTER

NOTE:

This is real enterprise architecture
Requires AWS billing account

Used in production environments
Not required for free lab experiments

EXPERIMENT NO: 4

Provision a VPS instance from a cloud provider or set up a virtual machine on your local system using software like VirtualBox or VMware. 2. Install a Linux distribution such as Ubuntu, CentOS, or Debian as the operating system. Configure network settings, including IP address, subnet mask, gateway, and DNS.

AIM

To provision a Virtual Private Server (VPS) using VirtualBox, install Ubuntu Server, configure network settings (IP Address, Subnet Mask, Gateway, DNS), and enable SSH remote access.

OBJECTIVES

1. To understand the concept of Virtual Private Server (VPS).
2. To create and manage virtual machines using VirtualBox.
3. To install Ubuntu Server OS.
4. To configure static network settings.
5. To enable SSH remote access.

SOFTWARE REQUIREMENTS

- Oracle VirtualBox
- Ubuntu Server 22.04 LTS ISO
- Windows/Linux Host System
- Internet Connection
- Terminal Access

THEORY

A Virtual Private Server (VPS) is a software-based server created using virtualization technology. It operates independently with its own operating system, storage, memory, and network configuration while sharing the physical hardware with other virtual machines.

Virtualization allows efficient utilization of resources and forms the foundation of cloud computing platforms.

Network configuration parameters include:

IP Address

A unique address assigned to a device on a network.

Subnet Mask

Determines the network and host portions of an IP address.

Gateway

Provides access to networks outside the local network.

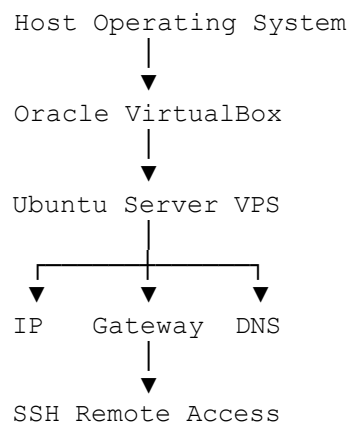
DNS (Domain Name System)

Converts domain names into IP addresses.

SSH (Secure Shell)

A secure protocol used for remote administration of Linux systems.

ARCHITECTURE



ALGORITHM

1. Install Oracle VirtualBox.
2. Create a new virtual machine.
3. Attach Ubuntu Server ISO image.
4. Install Ubuntu Server.
5. Update system packages.
6. Identify network interface.
7. Configure static IP address.
8. Configure subnet mask.
9. Configure default gateway.
10. Configure DNS server.

11. Apply network settings.
12. Verify connectivity.
13. Install OpenSSH Server.
14. Enable SSH service.
15. Access the VPS remotely using SSH.

PROCEDURE

Step 1: Install VirtualBox

Download and install Oracle VirtualBox.

Step 2: Create Virtual Machine

Configure:

Parameter	Value
Name	VPS-Server
OS Type	Ubuntu (64-bit)
RAM	2 GB
CPU	2 Cores
Storage	20 GB

Step 3: Install Ubuntu Server

1. Attach Ubuntu Server ISO.
2. Start VM.
3. Follow installation wizard.
4. Create username and password.
5. Complete installation.

Step 4: Update Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

Step 5: Identify Network Interface

```
ip a
```

Sample Interface:

```
ens33
```

```
2116231801176
```

Step 6: Configure Static IP

Edit Netplan file:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Add:

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Apply configuration:

```
sudo netplan apply
```

Step 7: Verify Network Configuration

Check IP:

```
ip addr
```

Check routing table:

```
ip route
```

Check DNS:

```
cat /etc/resolv.conf
```

Step 8: Test Connectivity

Test Internet:

```
ping -c 4 8.8.8.8
```

Test DNS:

```
ping -c 4 google.com
```

2116231801176

Step 9: Install SSH Server

```
sudo apt install openssh-server -y
```

Step 10: Enable SSH Service

```
sudo systemctl enable ssh  
sudo systemctl start ssh
```

Verify status:

```
sudo systemctl status ssh
```

Step 11: Connect Remotely

From another system:

```
ssh username@192.168.1.100
```

COMMANDS USED

```
sudo apt update  
sudo apt upgrade -y  
ip a  
sudo nano /etc/netplan/00-installer-config.yaml  
sudo netplan apply  
ip addr  
ip route  
cat /etc/resolv.conf  
ping -c 4 8.8.8.8  
ping -c 4 google.com  
sudo apt install openssh-server -y  
sudo systemctl enable ssh  
sudo systemctl start ssh  
sudo systemctl status ssh  
ssh username@192.168.1.100
```

SAMPLE OUTPUT

IP Configuration

```
inet 192.168.1.100/24
```

Routing Table

```
default via 192.168.1.1 dev ens33
```

```
2116231801176
```

DNS Configuration

```
nameserver 8.8.8.8
nameserver 8.8.4.4
```

SSH Status

Active: active (running)

OBSERVATIONS

Parameter	Status
VM Creation	Successful
Ubuntu Installation	Successful
Static IP Configuration	Successful
Subnet Mask Configuration	Successful
Gateway Configuration	Successful
DNS Configuration	Successful
Internet Connectivity	Verified
SSH Installation	Successful
SSH Service Running	Verified
Remote Login	Successful

PRECAUTIONS

1. Allocate sufficient RAM and storage to the VM.
2. Verify correct network adapter settings in VirtualBox.
3. Ensure the IP address belongs to the local network.
4. Avoid IP conflicts with other devices.
5. Verify DNS settings before testing connectivity.
6. Use strong passwords for SSH access.

APPLICATIONS

1. Web Hosting
2. Cloud Computing
3. Remote Administration
4. Database Hosting
5. Application Deployment
6. Development and Testing Environments

RESULT

A Virtual Private Server (VPS) was successfully created using Oracle VirtualBox. Ubuntu Server was installed and configured with static IP address, subnet mask, gateway, and DNS settings. SSH service was installed, enabled, and verified for secure remote access.

LEARNING OUTCOMES

After completing this experiment, the student is able to:

1. Create and manage virtual machines.
2. Install Ubuntu Server operating system.
3. Configure static network parameters.
4. Verify network connectivity.
5. Configure and manage SSH services.
6. Access Linux servers remotely.

VIVA QUESTIONS AND ANSWERS

1. What is VPS?

A VPS is a virtual machine that behaves like an independent server.

2. What is virtualization?

It is the creation of virtual computing resources using software.

3. What is VirtualBox?

A hypervisor used to create and manage virtual machines.

4. What is an IP address?

A unique address assigned to a device in a network.

5. What is a subnet mask?

It separates the network and host portions of an IP address.

6. What is a gateway?

A device used to connect different networks.

7. What is DNS?

A system that converts domain names into IP addresses.

8. What is SSH?

A secure protocol for remote login.

9. Why is a static IP used?

To maintain a fixed address for servers.

10. What is the default SSH port?

Port 22.

11. Which package provides SSH service in Ubuntu?

OpenSSH Server.

12. What command displays IP addresses?

```
ip a
```

13. What command applies Netplan configuration?

```
sudo netplan apply
```

14. What command displays routing information?

```
ip route
```

15. What command is used for remote login?

```
ssh username@IP_Address
```

16. What is the purpose of `/etc/resolv.conf`?

It stores DNS resolver information.

17. What is a network interface?

A hardware or virtual component that connects a device to a network.

18. What is the difference between static and dynamic IP?

Static IP remains fixed; dynamic IP changes automatically.

19. Why is SSH preferred over Telnet?

SSH encrypts communication, while Telnet does not.

20. What are the advantages of VPS?

Isolation, scalability, flexibility, and cost effectiveness.

EXPERIMENT NO: 5

CLOUD MIGRATION AND BACKUP SETUP USING AMAZON S3

AIM

To perform cloud migration and backup operations by storing backup files in Amazon S3, automating backup schedules, and restoring data to verify integrity.

OBJECTIVES

- To understand cloud migration concepts.
- To understand cloud-based backup solutions.
- To create and configure Amazon S3 storage.
- To upload and restore backup data from the cloud.
- To automate backup operations.
- To verify data integrity after restoration.

SOFTWARE REQUIREMENTS

- AWS Account
- Amazon S3 Service
- Ubuntu Linux Server / Virtual Machine
- AWS CLI
- Internet Connection
- Terminal Access

THEORY

Cloud Migration is the process of moving data, applications, and workloads from local infrastructure to cloud environments.

Cloud Backup is the process of storing copies of data in cloud storage services to ensure recovery in case of hardware failure, accidental deletion, cyber-attacks, or disasters. Amazon S3 (Simple Storage Service) is a highly scalable object storage service provided by AWS. It offers durability, availability, security, and lifecycle management features suitable for backup and archival purposes. In this experiment, local files are backed up and uploaded to Amazon S3. The backup process is automated using Linux scheduling tools and data is restored to verify successful recovery.

ARCHITECTURE

Source Data



Backup Archive Creation



AWS CLI



Amazon S3 Bucket



Cloud Storage



Restore Process



Data Verification

ALGORITHM

1. Create AWS account.
2. Create S3 bucket.
3. Install AWS CLI.
4. Configure AWS credentials.
5. Create backup archive.
6. Upload backup to Amazon S3.
7. Verify uploaded files.

8. Restore files from S3.
9. Verify data integrity.
10. Automate backup using cron.

FLOWCHART

Start



Create S3 Bucket



Install AWS CLI



Configure AWS Credentials



Create Backup



Upload to S3



Verify Upload



Restore Data



Verify Integrity



Schedule Automated Backup



2116231801176

Stop

PROCEDURE

Step 1: Login to AWS Console

1. Login to AWS Management Console.
2. Navigate to Amazon S3.
3. Open S3 Dashboard.

Step 2: Create S3 Bucket

1. Click **Create Bucket**.
2. Enter bucket name:

cloud-backup-lab-bucket

3. Select Region.
4. Keep default settings.
5. Click **Create Bucket**.

Step 3: Install AWS CLI

Update system:

```
sudo apt update
```

Install AWS CLI:

```
sudo apt install awscli -y
```

Verify installation:

```
aws --version
```

Step 4: Configure AWS CLI

```
aws configure
```

Provide:

- AWS Access Key ID
- AWS Secret Access Key
- Region Name
- Output Format

2116231801176

Step 5: Create Sample Data

```
mkdir migration_data  
cd migration_data  
echo "Cloud Computing Lab Backup" > file1.txt  
echo "AWS S3 Backup Experiment" > file2.txt
```

Step 6: Create Backup Archive

```
tar -czvf backup.tar.gz migration_data
```

Step 7: Upload Backup to Amazon S3

```
aws s3 cp backup.tar.gz s3://cloud-backup-lab-bucket/
```

Step 8: Verify Upload

```
aws s3 ls s3://cloud-backup-lab-bucket/
```

Expected Output:

```
backup.tar.gz
```

Step 9: Simulate Restore Operation

Create restore directory:

```
mkdir restore_data
```

Download backup from S3:

```
aws s3 cp s3://cloud-backup-lab-bucket/backup.tar.gz .
```

Extract backup:

```
tar -xzf backup.tar.gz -C restore_data
```

Step 10: Verify Restored Files

```
ls restore_data/migration_data
```

Expected Output:

```
file1.txt  
file2.txt
```

Step 11: Automate Backup Process

Create backup script:

```
nano backup_script.sh
```

Script Content:

```
#!/bin/bash

tar -czvf backup.tar.gz migration_data

aws s3 cp backup.tar.gz s3://cloud-backup-lab-bucket/
```

Provide permission:

```
chmod +x backup_script.sh
```

Open cron editor:

```
crontab -e
```

Schedule daily backup:

```
0 10 * * * /home/user/backup_script.sh
```

COMMANDS USED

```
aws --version

aws configure

aws s3 ls

aws s3 cp

tar -czvf

tar -xzvf

chmod +x

crontab -e
```

OBSERVATIONS

Parameter	Status
AWS Account Creation	Successful

2116231801176

Parameter	Status
S3 Bucket Creation	Successful
AWS CLI Installation	Successful
AWS Configuration	Successful
Backup Creation	Successful
Upload to S3	Successful
Restore from S3	Successful
Data Verification	Successful
Automation Setup	Successful

EXPECTED OUTPUT

- Amazon S3 bucket created successfully.
- Backup archive uploaded to cloud storage.
- Files restored successfully from S3.
- Data integrity verified.
- Automated backup schedule configured.

ADVANTAGES

- Highly durable cloud storage.
- Secure and scalable backup solution.
- Automated backup support.
- Disaster recovery capability.
- Global accessibility.

APPLICATIONS

- Enterprise backup systems.
- Disaster recovery solutions.
- Cloud migration projects.

- Long-term archival storage.
- Business continuity planning.

RESULT

A cloud migration and backup solution was successfully implemented using Amazon S3. Backup files were uploaded to cloud storage, restored successfully, and automated backup scheduling was configured.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Create and manage Amazon S3 buckets.
- Upload data to cloud storage.
- Restore backup data from cloud storage.
- Automate cloud backup operations.
- Understand cloud migration and disaster recovery concepts.

VIVA QUESTIONS

1. What is Amazon S3?

Amazon S3 is AWS object storage used for storing and retrieving data from the cloud.

2. What is cloud migration?

Cloud migration is the movement of data and applications from local systems to cloud platforms.

3. What is cloud backup?

Cloud backup is storing copies of data in cloud storage for recovery purposes.

4. What is AWS CLI?

AWS CLI is a command-line tool used to manage AWS services.

5. What is an S3 bucket?

An S3 bucket is a container used to store objects in Amazon S3.

6. How is data restored from S3?

Data can be downloaded using AWS CLI commands and extracted locally.

7. Why automate backups?

Automation ensures regular backups without manual intervention.

8. What are the benefits of cloud backup?

Scalability, durability, availability, security, and disaster recovery.

9. What is disaster recovery?

The process of recovering systems and data after failures or disasters.

10. What is the difference between local backup and cloud backup?

Local backups are stored on local devices, whereas cloud backups are stored on remote cloud infrastructure.

Experiment No: 6

CDN SERVICE SELECTION AND CONFIGURATION USING AMAZON CLOUDFRONT

AIM

To select a Content Delivery Network (CDN) provider, configure a CDN distribution, set up DNS records, and deliver web content through the CDN network using Amazon CloudFront.

OBJECTIVES

- To understand the concept of Content Delivery Networks (CDN).
- To study CDN providers and their features.
- To configure Amazon CloudFront as a CDN service.
- To configure DNS records for CDN integration.
- To create and manage CDN distributions.
- To verify content delivery through CDN edge locations.

SOFTWARE REQUIREMENTS

- AWS Account
- Amazon S3
- Amazon CloudFront
- Domain Name (Optional)
- Web Browser
- Internet Connection

THEORY

A Content Delivery Network (CDN) is a distributed network of servers that delivers web content from locations closest to users. CDN services reduce latency, improve website performance, enhance availability, and provide security features.

Popular CDN providers include:

- Cloudflare
- Amazon CloudFront
- Akamai
- Microsoft Azure CDN
- Google Cloud CDN

Amazon CloudFront is AWS's CDN service that distributes content globally through edge locations. It integrates seamlessly with Amazon S3, EC2, Load Balancers, and custom origins.

DNS (Domain Name System) configuration is essential in CDN deployment. DNS records map domain names to CDN distributions, allowing users to access content through custom domains.

CDN PROVIDER COMPARISON

Provider	Features
Cloudflare	Free plan, DNS management, DDoS protection
Amazon CloudFront	Global AWS integration, scalable CDN
Akamai	Enterprise-grade CDN services
Azure CDN	Microsoft ecosystem integration
Google Cloud CDN	Integrated with Google Cloud

ARCHITECTURE

User Browser



DNS Resolution



Amazon CloudFront Distribution



Edge Location

↓

Amazon S3 Bucket (Origin)

↓

Content Delivery

ALGORITHM

1. Create AWS account.
2. Create S3 bucket.
3. Upload website content.
4. Configure S3 permissions.
5. Create CloudFront distribution.
6. Select S3 bucket as origin.
7. Configure CDN settings.
8. Configure DNS records.
9. Access content through CloudFront URL.
10. Verify CDN delivery.

FLOWCHART

Start

↓

Create S3 Bucket

↓

Upload Content

↓

Create CloudFront Distribution

↓

Configure Origin

↓

Configure DNS

↓

Deploy Distribution

↓

Access CDN URL

↓

Verify Content Delivery

↓

Stop

PROCEDURE

Step 1: Login to AWS Console

1. Open AWS Management Console.
2. Login using AWS credentials.
3. Navigate to Amazon S3.

Step 2: Create S3 Bucket

1. Click **Create Bucket**.
2. Enter bucket name:

cdn-lab-bucket

3. Select Region.
4. Click **Create Bucket**.

Step 3: Upload Website Content

Create sample webpage:

```
<html>
<head>
<title>CDN Experiment</title>
</head>
<body>
<h1>Welcome to CloudFront CDN</h1>
</body>
</html>
```

2116231801176

Save as:

`index.html`

Upload file to S3 bucket.

Step 4: Enable Static Website Hosting

1. Open S3 Bucket.
2. Navigate to **Properties**.
3. Enable **Static Website Hosting**.
4. Set:

Index Document: `index.html`

5. Save changes.

Step 5: Configure Bucket Permissions

1. Navigate to **Permissions**.
2. Configure bucket policy to allow CloudFront access.
3. Save configuration.

Step 6: Create CloudFront Distribution

1. Open Amazon CloudFront Console.
2. Click **Create Distribution**.
3. Under Origin Domain select:

`cdn-lab-bucket.s3.amazonaws.com`

4. Configure default settings.
5. Enable caching.
6. Click **Create Distribution**.

Step 7: Wait for Distribution Deployment

CloudFront creates a unique distribution URL:

Example:

`d123abcdxyz.cloudfront.net`

Deployment may take several minutes.

Step 8: Test CDN Distribution

2116231801176

Open browser:

`https://d123abcdxyz.cloudfront.net`

Verify webpage loads successfully.

Step 9: Configure Custom Domain (Optional)

Suppose domain:

`www.example.com`

Open DNS management panel.

Create CNAME record:

Type	Name	Value
CNAME	www	d123abcdxyz.cloudfront.net

Save DNS configuration.

Step 10: Verify DNS Resolution

Check DNS mapping:

`nslookup www.example.com`

OR

`dig www.example.com`

Verify CloudFront distribution is returned.

Step 11: Verify CDN Delivery

Access:

`https://www.example.com`

Verify content is served successfully through CloudFront.

Step 12: View Distribution Statistics

1. Open CloudFront Console.
2. Select Distribution.
3. View:

2116231801176

- Requests
- Data Transfer
- Cache Hit Ratio
- Geographic Distribution

COMMANDS USED

nslookup www.example.com

dig www.example.com

AWS Services Used:

- Amazon S3
- Amazon CloudFront
- Route 53 (Optional)

OBSERVATIONS

Parameter	Status
AWS Account Creation	Successful
S3 Bucket Creation	Successful
Website Upload	Successful
Static Hosting Enabled	Successful
CloudFront Distribution Creation	Successful
DNS Configuration	Successful
CDN URL Access	Successful
Content Delivery Verification	Successful

EXPECTED OUTPUT

- Website content uploaded successfully.
- CloudFront distribution created.
- DNS records configured.

- CDN URL accessible.
- Content delivered through CDN edge locations.
- Reduced latency and improved performance.

ADVANTAGES

- Faster content delivery.
- Reduced latency.
- Global availability.
- Improved website performance.
- DDoS protection support.
- Scalable architecture.

APPLICATIONS

- Website Acceleration.
- Video Streaming.
- Software Downloads.
- E-Commerce Websites.
- Web Applications.
- API Content Delivery.

RESULT

A Content Delivery Network was successfully implemented using Amazon CloudFront. Website content was stored in Amazon S3, distributed through CloudFront edge locations, and accessed through CDN URLs and DNS configuration.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Select an appropriate CDN provider.
- Configure Amazon CloudFront distributions.
- Configure DNS records for CDN integration.
- Connect S3 storage with CloudFront.
- Verify content delivery through CDN edge locations.
- Analyze CDN performance and availability.

VIVA QUESTIONS

1. What is a CDN?

A CDN is a distributed network of servers that delivers content from locations nearest to users.

2. What is Amazon CloudFront?

Amazon CloudFront is AWS's Content Delivery Network service.

3. Why is CDN used?

CDNs reduce latency, improve performance, and increase availability.

4. What is an Edge Location?

An edge location is a CDN server that stores cached content closer to end users.

5. What is a CloudFront Distribution?

A CloudFront Distribution is a configuration that defines how content is delivered through CloudFront.

6. What is an Origin Server?

The origin server is the source location from which CDN retrieves content.

7. What is DNS?

DNS translates domain names into IP addresses or service endpoints.

8. What is a CNAME Record?

A CNAME record maps one domain name to another domain name.

9. What is Route 53?

Route 53 is AWS's managed DNS service.

10. What are the advantages of CloudFront?

Low latency, global reach, scalability, security, and high availability.

ADDITIONAL EXPERIMENTS

EXPERIMENT NO: 7

CONTAINERIZATION AND APPLICATION DEPLOYMENT USING DOCKER

AIM

To install Docker, create a containerized web application environment using a Dockerfile, and manage the container lifecycle.

OBJECTIVES

- To understand the concept of containerization and Docker.
- To install and configure Docker Engine.
- To create custom Docker images using Dockerfile.
- To deploy applications using Docker containers.
- To manage container lifecycle operations.
- To verify containerized application deployment.

SOFTWARE REQUIREMENTS

- Ubuntu Server 22.04 LTS
- Docker Engine (Community Edition)
- Text Editor (Nano/Vim)
- Web Browser
- Internet Connection

THEORY

Containerization is a lightweight virtualization technology that enables applications and their dependencies to be packaged together and executed consistently across different computing environments. Unlike traditional virtual machines, containers share the host operating system kernel, resulting in reduced resource consumption and faster startup times.

Docker is an open-source containerization platform that simplifies application deployment by packaging software into standardized units called containers.

Key Components:

- **Docker Engine** – Core service responsible for building and running containers.
- **Docker Image** – Read-only template used to create containers.
- **Docker Container** – Running instance of a Docker image.
- **Dockerfile** – Script containing instructions to build Docker images.
- **Docker Hub** – Online repository for Docker images.

ARCHITECTURE

Developer Application



Dockerfile



Docker Image



Docker Container



Application Service

ALGORITHM

1. Install Docker Engine.
2. Verify Docker installation.
3. Create application directory.
4. Create sample web application.
5. Create Dockerfile.
6. Build Docker image.
7. Run Docker container.
8. Verify container status.
9. Access application through browser.
10. View logs and container information.
11. Stop and remove container.

FLOWCHART

Start



Install Docker



Verify Installation



Create Application Files



Create Dockerfile



Build Docker Image

↓
Run Docker Container
↓
Verify Application
↓
View Logs
↓
Stop Container
↓
End

PROCEDURE

Step 1: Update System Packages

Execute:

```
sudo apt update
```

```
sudo apt upgrade -y
```

Step 2: Install Docker Engine

Execute:

```
sudo apt install docker.io -y
```

Step 3: Start Docker Service

Execute:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Step 4: Verify Docker Installation

Execute:

```
docker --version
```

Expected Output:

Docker version XX.X.X

Step 5: Create Application Directory

Execute:

```
mkdir docker-lab
```

```
cd docker-lab
```

Step 6: Create Sample Web Page

Create file:

```
index.html
```

Step 7: Create Dockerfile

Create file:

```
Dockerfile
```

Step 8: Build Docker Image

Execute:

```
docker build -t cloud-app-img .
```

Step 9: Verify Image Creation

Execute:

```
docker images
```

Step 10: Run Docker Container

Execute:

```
docker run -d -p 8080:80 --name web-container cloud-app-img
```

Step 11: Verify Running Container

Execute:

```
docker ps
```

Step 12: Access Application

2116231801176

Open Browser:

<http://localhost:8080>

Step 13: View Container Logs

Execute:

```
docker logs web-container
```

Step 14: Stop Container

Execute:

```
docker stop web-container
```

Step 15: Remove Container

Execute:

```
docker rm web-container
```

COMMANDS USED

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
docker --version
```

```
docker build -t cloud-app-img .
```

```
docker images
```

```
docker run -d -p 8080:80 --name web-container cloud-app-img
```

```
docker ps
```

```
docker logs web-container
```

2116231801176

docker stop web-container

docker rm web-container

OBSERVATIONS

Parameter	Status
Docker Installation	Successful
Docker Service Startup	Successful
Docker Image Build	Successful
Container Deployment	Successful
Port Mapping	Successful
Application Access	Successful
Container Logs	Verified

EXPECTED OUTPUT

- Docker Engine installed successfully.
- Docker image created successfully.
- Container deployed successfully.
- Application accessible through browser.
- Container logs generated successfully.
- Container lifecycle operations completed successfully.

ADVANTAGES

- Lightweight virtualization.
- Faster deployment.
- Portability across platforms.
- Efficient resource utilization.
- Easy scalability.
- Consistent execution environment.

APPLICATIONS

- Cloud-native applications.
- Microservices architecture.
- DevOps pipelines.
- Web application deployment.
- Continuous Integration/Continuous Deployment (CI/CD).
- Software testing environments.

RESULT

A Docker-based containerized application environment was successfully created. A custom Docker image was built using a Dockerfile and deployed as a running container. The application was successfully accessed and managed through Docker commands.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Install and configure Docker Engine.
- Create Docker images using Dockerfile.
- Deploy and manage Docker containers.
- Perform container lifecycle operations.
- Understand containerization concepts.
- Deploy applications using Docker technology.

VIVA QUESTIONS WITH ANSWERS

1. What is Docker?

Docker is an open-source containerization platform used to build, deploy, and manage applications in isolated containers.

2. What is a Container?

A container is a lightweight, standalone, executable package that contains an application along with all its dependencies, libraries, and configuration files.

3. What is Docker Engine?

Docker Engine is the core software that enables the creation, execution, and management of Docker containers.

4. What is a Docker Image?

A Docker Image is a read-only template containing application code, libraries, dependencies, and instructions required to create a Docker container.

5. What is a Dockerfile?

A Dockerfile is a text file containing a set of instructions used to automatically build a Docker Image.

6. What is Docker Hub?

Docker Hub is a cloud-based repository that stores and distributes Docker Images.

7. What is the difference between a Virtual Machine and a Container?

Virtual Machine	Container
Includes a complete operating system	Shares the host operating system kernel
Requires more resources	Requires fewer resources
Larger storage size	Lightweight
Slower startup	Faster startup
Hardware-level virtualization	OS-level virtualization

8. What command is used to build a Docker Image?

```
docker build -t image_name .
```

9. What command is used to run a Docker Container?

```
docker run image_name
```

10. What command is used to list running containers?

```
docker ps
```

11. What command is used to stop a running container?

```
docker stop container_name
```

12. What command is used to remove a container?

```
docker rm container_name
```

13. What command is used to view container logs?

```
docker logs container_name
```

14. What is Port Mapping in Docker?

Port mapping is the process of connecting a container port to a host machine port, enabling external access to services running inside the container.

Example:

```
docker run -p 8080:80 image_name
```

15. What is the purpose of the EXPOSE instruction in a Dockerfile?

The EXPOSE instruction informs Docker that a container listens on a specified network port during runtime.

16. What is the purpose of the COPY instruction in a Dockerfile?

The COPY instruction copies files and directories from the host system into the Docker Image.

17. What is the purpose of the CMD instruction in a Dockerfile?

The CMD instruction specifies the default command that runs when a container starts.

18. What is Containerization?

Containerization is a software deployment method in which applications and their dependencies are packaged into isolated containers for consistent execution across environments.

19. What are the advantages of Docker?

- Lightweight and fast deployment
- Portability across platforms
- Efficient resource utilization
- Easy scalability
- Simplified application deployment
- Consistent execution environment

20. What are the applications of Docker?

- Cloud-native applications
- Microservices architecture
- Continuous Integration/Continuous Deployment (CI/CD)
- DevOps automation
- Web application deployment
- Software testing environments

21. What is the difference between a Docker Image and a Docker Container?

A Docker Image is a template used to create containers, whereas a Docker Container is a running instance of that image.

22. Why is Docker widely used in Cloud Computing?

Docker enables rapid deployment, portability, scalability, resource efficiency, and consistent application execution across cloud environments.

23. What command is used to view available Docker Images?

```
docker images
```

24. What happens when a Docker Container is stopped?

The application inside the container stops executing, but the container configuration and data remain available unless the container is removed.

EXPERIMENT NO: 8

SERVERLESS COMPUTING IMPLEMENTATION USING AWS LAMBDA

AIM

To design, deploy, and execute a serverless compute function using AWS Lambda that processes data dynamically without provisioning underlying server infrastructure.

OBJECTIVES

- To understand the concept of Serverless Computing and Function-as-a-Service (FaaS).
- To create and configure AWS Lambda functions.
- To implement event-driven application logic.
- To execute and test Lambda functions using sample events.
- To monitor execution using Amazon CloudWatch.
- To understand serverless architecture deployment models.

SOFTWARE REQUIREMENTS

- AWS Free Tier Account
- AWS Lambda Service
- Amazon CloudWatch
- Web Browser
- Internet Connection

THEORY

Serverless Computing is a cloud computing execution model where the cloud provider manages the underlying infrastructure, operating system, scaling, and resource allocation automatically. Developers only focus on writing and deploying application code. AWS Lambda is a serverless compute service provided by Amazon Web Services that executes code in response to events and automatically manages the required infrastructure. Lambda functions can be triggered by services such as Amazon S3, API Gateway, DynamoDB, CloudWatch Events, and other AWS services.

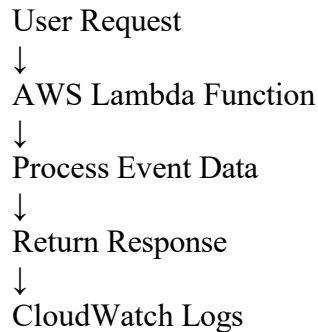
Key Concepts:

- **Function-as-a-Service (FaaS)** – Event-driven execution model.
- **Lambda Function** – Independent unit of executable code.
- **Event Trigger** – Action that invokes the Lambda function.

- **CloudWatch Logs** – Monitoring and logging service.
- **Execution Role** – IAM permissions assigned to Lambda.

AWS Lambda supports automatic scaling, fault tolerance, and pay-per-use billing.

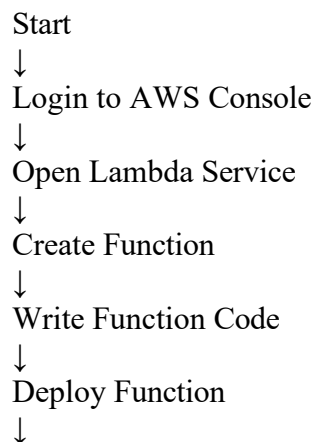
ARCHITECTURE



ALGORITHM

1. Login to AWS Management Console.
2. Navigate to AWS Lambda service.
3. Create a new Lambda function.
4. Configure runtime environment.
5. Write Lambda function code.
6. Deploy the function.
7. Create test event payload.
8. Execute Lambda function.
9. Verify output response.
10. Analyze CloudWatch logs.
11. Validate successful execution.

FLOWCHART



Create Test Event

↓

Execute Function

↓

View Output

↓

Check CloudWatch Logs

↓

Stop

PROCEDURE

Step 1: Login to AWS Console

1. Open AWS Management Console.
2. Enter AWS credentials.
3. Login successfully.

Step 2: Open AWS Lambda Service

1. Search for **Lambda**.
2. Select AWS Lambda Service.
3. Open Lambda Dashboard.

Step 3: Create Lambda Function

1. Click **Create Function**.
2. Select **Author from Scratch**.
3. Enter:

Function Name:

`student-lambda-function`

Runtime:

`Python 3.x`

4. Select **Create a New Execution Role**.
5. Click **Create Function**.

Step 4: Write Lambda Function Code

Replace existing code with:

```
import json
```

```
2116231801176
```

```
def lambda_handler(event, context):

    print("Received Event:", event)

    user_name = event.get('name', 'Cloud Student')

    return {
        'statusCode': 200,
        'body': json.dumps(
            f'Hello {user_name}! Serverless function executed successfully.'
        )
    }
```

Step 5: Deploy Function

1. Click **Deploy**.
2. Verify successful deployment message.

Step 6: Create Test Event

1. Click **Test**.
2. Create new test event.

Event Name:

student-test-event

JSON Payload:

```
{
  "name": "Surendar"
}
```

3. Save test event.

Step 7: Execute Function

1. Click **Test**.
2. Wait for execution.

Step 8: Verify Output

Expected Response:

```
{
  "statusCode": 200,
  "body": "\"Hello Surendar! Serverless function executed successfully.\""
}
```

2116231801176

Step 9: View CloudWatch Logs

1. Select **Monitor** Tab.
2. Click **View CloudWatch Logs**.
3. Verify execution details.

Step 10: Analyze Execution Metrics

Verify:

- Invocation Count
- Execution Duration
- Memory Usage
- Request Status

COMMANDS USED

Python Lambda Function:

```
import json

def lambda_handler(event, context):

    user_name = event.get('name', 'Cloud Student')

    return {
        'statusCode': 200,
        'body': json.dumps(
            f'Hello {user_name}! Serverless function executed successfully.'
        )
    }
```

Sample Event:

```
{
  "name": "Surendar"
}
```

OBSERVATIONS

Parameter	Status
Lambda Function Creation	Successful
Function Deployment	Successful
Test Event Creation	Successful
Event Processing	Successful
Function Execution	Successful

2116231801176

Parameter	Status
Response Generation	Successful
CloudWatch Logging	Verified

EXPECTED OUTPUT

- AWS Lambda function created successfully.
- Function deployed without errors.
- Test event processed successfully.
- Correct response generated.
- CloudWatch logs available.
- Serverless execution verified.

ADVANTAGES

- No server management required.
- Automatic scaling.
- Pay-per-use model.
- Reduced operational overhead.
- High availability.
- Faster deployment.

APPLICATIONS

- Backend APIs
- Event Processing
- Data Transformation
- File Processing
- IoT Applications
- Real-Time Analytics
- Cloud Automation

RESULT

A serverless application was successfully implemented using AWS Lambda. The Lambda function processed event data, generated dynamic responses, and executed without requiring server provisioning. CloudWatch logs verified successful execution and monitoring.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Understand Serverless Computing concepts.
- Create and deploy AWS Lambda functions.
- Configure event-driven execution.
- Execute and test Lambda functions.
- Monitor Lambda using CloudWatch.
- Implement Function-as-a-Service (FaaS) solutions.

VIVA QUESTIONS WITH ANSWERS

1. What is Serverless Computing?

Serverless Computing is a cloud execution model where the cloud provider manages infrastructure automatically and users focus only on application code.

2. What is AWS Lambda?

AWS Lambda is a serverless compute service that executes code in response to events.

3. What is Function-as-a-Service (FaaS)?

FaaS is a cloud model where individual functions are executed on demand without server management.

4. What is a Lambda Function?

A Lambda Function is a block of code executed by AWS Lambda in response to an event.

5. What is an Event Trigger?

An event trigger is an action that invokes a Lambda function.

6. What services can trigger Lambda?

Amazon S3, API Gateway, DynamoDB, CloudWatch Events, SNS, and SQS.

7. What is CloudWatch?

Amazon CloudWatch is a monitoring and logging service for AWS resources.

8. What is an Execution Role?

An IAM role that grants permissions to a Lambda function.

9. What runtime languages are supported by AWS Lambda?

Python, Java, Node.js, C#, Go, Ruby, and others.

10. What are the advantages of AWS Lambda?

Automatic scaling, reduced cost, high availability, and no server administration.

11. What is the default entry point in Python Lambda functions?

`lambda_handler()`

12. What is the purpose of the event parameter?

It contains input data passed to the Lambda function.

13. What is the purpose of the context parameter?

It provides runtime information about the function execution.

14. What is the main benefit of Serverless Computing?

Developers focus on application logic without managing infrastructure.

15. What are the applications of AWS Lambda?

APIs, automation, file processing, IoT systems, analytics, and event-driven applications.

EXPERIMENT NO: 9

CLOUD DATABASE PROVISIONING AND MANAGEMENT USING AMAZON RDS

AIM

To provision a fully managed Relational Database Service (RDS) instance using AWS Free Tier, configure security settings, and verify remote database connectivity.

OBJECTIVES

- To understand cloud-based managed database services.
- To create and configure an Amazon RDS database instance.
- To understand database security using Security Groups.
- To establish remote database connectivity.
- To perform basic database operations on a cloud-hosted database.
- To understand the advantages of managed database services.

SOFTWARE REQUIREMENTS

- AWS Free Tier Account
- Amazon RDS Service
- MySQL Client / MySQL Workbench / DBeaver
- Web Browser
- Internet Connection

THEORY

Amazon Relational Database Service (Amazon RDS) is a fully managed database service provided by AWS that simplifies the setup, operation, maintenance, and scaling of relational databases in the cloud.

RDS supports multiple database engines such as:

- MySQL
- PostgreSQL
- MariaDB
- Oracle Database
- Microsoft SQL Server

Amazon RDS automates routine database administration tasks including:

- Hardware provisioning
- Operating system updates
- Database patching
- Automated backups
- Monitoring and logging
- Disaster recovery

Security Groups act as virtual firewalls controlling access to the database instance.

ARCHITECTURE

Client Application



Security Group



Amazon RDS Instance



Cloud Database Storage

ALGORITHM

1. Login to AWS Management Console.
2. Navigate to Amazon RDS.
3. Create a database instance.
4. Configure database settings.
5. Select Free Tier template.
6. Configure connectivity settings.
7. Create Security Group.
8. Launch RDS instance.
9. Obtain database endpoint.
10. Configure inbound access rules.
11. Connect using database client.
12. Execute SQL commands.
13. Verify successful database operations.

FLOWCHART

Start



Login to AWS Console



Open Amazon RDS



Create Database



2116231801176

Configure Parameters
↓
Launch Database Instance
↓
Retrieve Endpoint
↓
Configure Security Group
↓
Connect Database Client
↓
Execute SQL Commands
↓
Verify Results
↓
End

PROCEDURE

Step 1: Login to AWS Console

1. Open AWS Management Console.
2. Enter AWS credentials.
3. Login successfully.

Step 2: Open Amazon RDS

1. Search for **RDS**.
2. Select **Amazon RDS**.
3. Open RDS Dashboard.

Step 3: Create Database

1. Click **Create Database**.
2. Select:

Standard Create

3. Choose Engine:

MySQL

4. Select Template:

Free Tier

Step 4: Configure Database Settings

2116231801176

Database Identifier:

cloud-lab-db

Master Username:

admin

Master Password:

password@123

Step 5: Configure Instance Settings

Select:

db.t3.micro

or Free Tier eligible instance type.

Step 6: Configure Connectivity

1. Select Default VPC.
2. Enable:

Public Access = Yes

3. Create New Security Group.

Step 7: Create Database

Click:

Create Database

Wait until database status changes to:

Available

Step 8: Retrieve Database Endpoint

Navigate to:

RDS → Databases → cloud-lab-db

Record:

2116231801176

- Endpoint URL
- Port Number

Example:

```
cloud-lab-db.xxxxx.us-east-1.rds.amazonaws.com
```

Port:

```
3306
```

Step 9: Configure Security Group

Navigate to:

EC2 → Security Groups

Add Inbound Rule:

Type	Port	Source
MySQL/Aurora	3306	My IP

Save changes.

Step 10: Connect Using MySQL Client

Execute:

```
mysql -h cloud-lab-db.xxxxx.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
```

Enter password when prompted.

Step 11: Create Database

Execute:

```
CREATE DATABASE cloud_lab_db;
```

Step 12: Select Database

Execute:

```
USE cloud_lab_db;
```

Step 13: Create Table

2116231801176

Execute:

```
CREATE TABLE student(  
id INT PRIMARY KEY,  
name VARCHAR(50)  
);
```

Step 14: Insert Data

Execute:

```
INSERT INTO student VALUES  
(1, 'Surendar');
```

Step 15: Verify Data

Execute:

```
SELECT * FROM student;
```

Expected Output:

```
+-----+-----+  
| id | name      |  
+-----+-----+  
| 1  | Surendar |  
+-----+-----+
```

Step 16: Cleanup Resources

After completing the experiment:

1. Delete test database objects.
2. Delete the RDS instance.
3. Verify instance removal.

This prevents charges beyond AWS Free Tier limits.

COMMANDS USED

MySQL Connection

```
mysql -h cloud-lab-db.xxxxx.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
```

Database Commands

```
CREATE DATABASE cloud_lab_db;
```

2116231801176

```
USE cloud_lab_db;

CREATE TABLE student(
id INT PRIMARY KEY,
name VARCHAR(50)
);

INSERT INTO student VALUES
(1, 'Surendar');

SELECT * FROM student;

SHOW DATABASES;

SHOW TABLES;
```

OBSERVATIONS

Parameter	Status
RDS Instance Creation	Successful
Database Endpoint Generation	Successful
Security Group Configuration	Successful
Remote Connectivity	Successful
Database Creation	Successful
Table Creation	Successful
Data Insertion	Successful
Query Execution	Successful

EXPECTED OUTPUT

- Amazon RDS instance created successfully.
- Database endpoint generated successfully.
- Security Group configured correctly.
- Remote connection established successfully.
- Database operations executed successfully.
- Data retrieved successfully.

ADVANTAGES

- Fully managed database service.
- Automatic backups.
- High availability.
- Automatic software updates.
- Enhanced security.
- Easy scalability.

APPLICATIONS

- Web Applications
- Enterprise Applications
- E-Commerce Systems
- Content Management Systems
- Customer Relationship Management (CRM) Systems
- Cloud-Native Applications

RESULT

An Amazon RDS database instance was successfully provisioned and configured using AWS Free Tier resources. Secure remote connectivity was established, and database operations such as creation, insertion, and retrieval of data were performed successfully.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Create and configure Amazon RDS instances.
- Configure database security using Security Groups.
- Connect remotely to cloud databases.
- Perform SQL operations on cloud-hosted databases.
- Understand managed database services.
- Deploy databases in cloud environments.

VIVA QUESTIONS WITH ANSWERS

1. What is Amazon RDS?

Amazon RDS is a fully managed relational database service provided by AWS.

2. What does RDS stand for?

Relational Database Service.

3. What databases are supported by Amazon RDS?

MySQL, PostgreSQL, MariaDB, Oracle, and Microsoft SQL Server.

4. What is the main advantage of Amazon RDS?

It automates database administration tasks such as backups, updates, and scaling.

5. What is a Security Group?

A Security Group is a virtual firewall that controls inbound and outbound traffic.

6. What is an RDS Endpoint?

An endpoint is the hostname used to connect to an RDS instance.

7. Which port is commonly used for MySQL?

Port 3306.

8. What is the purpose of Public Access in RDS?

It allows external clients to connect to the database.

9. What is the purpose of automated backups?

To recover data in case of failures or accidental deletion.

10. What is a managed database service?

A database service where administration and maintenance tasks are handled by the cloud provider.

11. What command is used to show databases in MySQL?

```
SHOW DATABASES;
```

12. What command is used to display tables?

```
SHOW TABLES;
```

13. What command is used to retrieve records from a table?

```
SELECT * FROM student;
```

14. Why should the RDS instance be deleted after the experiment?

To prevent unnecessary resource consumption and charges.

15. What are the applications of Amazon RDS?

Web applications, enterprise systems, e-commerce platforms, and cloud-native applications.

EXPERIMENT NO: 10

MULTI-CONTAINER ORCHESTRATION USING DOCKER COMPOSE

AIM

To define, deploy, and manage a multi-container application consisting of a web frontend and database backend using Docker Compose.

OBJECTIVES

- To understand multi-container application architecture.
- To study microservices-based deployment models.
- To install and configure Docker Compose.
- To create and manage multiple containers using a single YAML configuration file.
- To establish communication between containers.
- To perform deployment, monitoring, and cleanup operations using Docker Compose.

SOFTWARE REQUIREMENTS

- Ubuntu Server 22.04 LTS
- Docker Engine
- Docker Compose Plugin
- Text Editor (Nano/Vim)
- Web Browser
- Internet Connection

THEORY

Modern cloud applications are commonly designed using a microservices architecture where individual services perform dedicated tasks. Each service is deployed independently and communicates with other services through network interfaces.

Managing multiple containers manually can be complex and error-prone. Docker Compose simplifies multi-container management by allowing developers to define services, networks, volumes, and dependencies using a YAML configuration file.

Docker Compose provides:

- Simplified multi-container deployment.
- Automatic network creation.

- Service dependency management.
- Centralized configuration management.
- Easy scaling and maintenance.

In this experiment, a WordPress web application is connected to a MySQL database backend using Docker Compose.

ARCHITECTURE

User Browser



WordPress Container



Docker Network



MySQL Container



Persistent Database Storage

ALGORITHM

1. Install Docker Engine.
2. Install Docker Compose.
3. Create project directory.
4. Create Docker Compose configuration file.
5. Define database service.
6. Define web application service.
7. Configure service dependencies.
8. Launch multi-container stack.
9. Verify running services.
10. Access application through browser.
11. View logs and status.
12. Stop and remove services.

FLOWCHART

Start



Install Docker Compose



Create Project Directory



Create docker-compose.yml



Define Services

↓
Launch Containers
↓
Verify Status
↓
Access Application
↓
View Logs
↓
Stop Services
↓
End

PROCEDURE

Step 1: Verify Docker Installation

Execute:

```
docker --version
```

Verify Docker is installed successfully.

Step 2: Verify Docker Compose Installation

Execute:

```
docker compose version
```

Expected Output:

```
Docker Compose version v2.x.x
```

Step 3: Create Project Directory

Execute:

```
mkdir compose-lab  
cd compose-lab
```

Step 4: Create Docker Compose File

Create file:

```
docker-compose.yml
```

Step 5: Configure Multi-Container Architecture

Add the following configuration:

```
services:

  db_backend:
    image: mysql:8.0
    container_name: compose-db-service

    environment:
      MYSQL_ROOT_PASSWORD: lab_secure_password
      MYSQL_DATABASE: student_records

    ports:
      - "3306:3306"

  web_frontend:
    image: wordpress:latest
    container_name: compose-web-service

    ports:
      - "8080:80"

    environment:
      WORDPRESS_DB_HOST: db_backend:3306
      WORDPRESS_DB_PASSWORD: lab_secure_password

    depends_on:
      - db_backend
```

Save the file.

Step 6: Validate Configuration File

Execute:

```
docker compose config
```

Verify no syntax errors are reported.

Step 7: Launch Multi-Container Application

Execute:

```
docker compose up -d
```

Docker Compose will:

- Pull required images.
- Create Docker network.

2116231801176

- Start MySQL container.
- Start WordPress container.

Step 8: Verify Running Containers

Execute:

```
docker compose ps
```

Expected Output:

Both containers should display:

Up

status.

Step 9: Verify Network Creation

Execute:

```
docker network ls
```

Verify Docker Compose network is created automatically.

Step 10: Access Application

Open Browser:

```
http://localhost:8080
```

Verify WordPress installation page appears.

Step 11: View Container Logs

Execute:

```
docker compose logs
```

View logs for all services.

Step 12: View Individual Service Logs

Database Logs:

```
docker compose logs db_backend
```

2116231801176

Web Service Logs:

```
docker compose logs web_frontend
```

Step 13: Stop Services

Execute:

```
docker compose stop
```

Step 14: Remove Containers and Network

Execute:

```
docker compose down
```

Verify containers and network are removed.

COMMANDS USED

Docker Verification

```
docker --version
docker compose version
```

Deployment Commands

```
docker compose up -d
docker compose ps
docker compose logs
```

Network Verification

```
docker network ls
```

Shutdown Commands

```
docker compose stop
docker compose down
```

OBSERVATIONS

Parameter	Status
Docker Compose Installation	Successful
YAML Configuration Validation	Successful

Parameter	Status
MySQL Container Deployment	Successful
WordPress Container Deployment	Successful
Automatic Network Creation	Successful
Service Dependency Resolution	Verified
Application Accessibility	Successful
Service Logs Generation	Verified
Stack Shutdown	Successful

EXPECTED OUTPUT

- Docker Compose configuration validated successfully.
- MySQL container started successfully.
- WordPress container started successfully.
- Automatic networking established between services.
- WordPress installation page accessible through browser.
- Multi-container orchestration completed successfully.

ADVANTAGES

- Simplified multi-container deployment.
- Automatic network management.
- Centralized configuration.
- Easy scalability.
- Faster development and testing.
- Reduced operational complexity.

APPLICATIONS

- Microservices Architecture
- Web Application Deployment
- Cloud-Native Applications
- DevOps Automation
- Continuous Integration and Continuous Deployment (CI/CD)
- Development and Testing Environments

RESULT

A multi-container application consisting of a WordPress frontend and MySQL backend was successfully deployed and managed using Docker Compose. Automatic service networking, dependency handling, monitoring, and cleanup operations were performed successfully.

LEARNING OUTCOME

After completing this experiment, the student is able to:

- Understand Docker Compose architecture.
- Create multi-container applications.
- Configure service dependencies.
- Manage Docker networks automatically.
- Deploy and monitor multiple containers.
- Implement microservices-based application deployment.

VIVA QUESTIONS WITH ANSWERS

1. What is Docker Compose?

Docker Compose is a tool used to define and manage multi-container Docker applications using a YAML configuration file.

2. What is the purpose of Docker Compose?

It simplifies deployment, networking, and management of multiple containers.

3. What file is used by Docker Compose?

docker-compose.yml

4. What format is used in Docker Compose configuration files?

YAML (Yet Another Markup Language).

5. What command is used to start services?

```
docker compose up -d
```

6. What command is used to stop services?

```
docker compose stop
```

7. What command is used to remove services and networks?

```
docker compose down
```

8. What is the purpose of depends_on?

It defines service startup dependencies.

9. What is a microservice?

A microservice is an independently deployable software component performing a specific business function.

10. How do containers communicate in Docker Compose?

Through automatically created Docker networks.

11. What is the purpose of docker compose logs?

To view logs generated by services.

12. What is automatic network creation in Docker Compose?

Docker Compose automatically creates a dedicated network allowing services to communicate.

13. What is the advantage of multi-container architecture?

It improves modularity, scalability, and maintainability.

14. What are common applications of Docker Compose?

Microservices, web applications, testing environments, and DevOps workflows.

15. What is the main benefit of Docker Compose over manual Docker commands?

It manages multiple containers, networks, and dependencies using a single configuration file and command.